Natural Language Processing (Fall-2026)

Assignment-2

Instructor: Dr. Hajra Waheed

Teacher Assistant

- **Sidra Saleem:** l258009@lhr.nu.edu.pk
- For **assignment & Viva-related queries**, please contact the TA directly via email.

Submission Guidelines:

- Submit your assignment on Google Classroom in the format "20XX.ipynb".
- **The deadline is September 24, 2026, at 11:59 PM. No extensions will be granted.**

Declarations:

- **Plagiarism will result in zero marks for the assignment.**
- This is an individual assignment; collaboration or group work is strictly prohibited.
- Please ensure that you submit your own original work.

VIVA Policy:

- A VIVA (oral examination) will be conducted to assess your understanding of the assignment.
- The VIVA will be scheduled separately, and you will be notified of the date and time.
- Failure to attend the VIVA will result in zero marks for the assignment.

Academic Integrity:

- Plagiarism, collusion, and academic dishonesty will not be tolerated.
- Any instances of academic misconduct will be reported to the authorities and may result in severe penalties.

Objective

This assignment builds upon the foundational NLP skills developed in Assignment 1. Students will explore advanced smoothing techniques for language models, tackle the out-of-vocabulary problem, and implement probabilistic text classification from scratch.

Through hands-on implementation, students will:

- Implement and analyze advanced smoothing techniques (Good-Turing and Kneser-Ney) for n-gram language models and compare them against Laplace smoothing from Assignment 1
- Investigate out-of-vocabulary (OOV) handling strategies and their impact on language model evaluation
- Build a Multinomial Naive Bayes text classifier from scratch for multi-class classification
- Conduct systematic ablation studies to understand how hyperparameters and design choices affect model performance

By the end of this assignment, students will have a deep understanding of how smoothing addresses data sparsity in language models, how OOV words affect model evaluation, and how probabilistic classifiers leverage text features for real-world classification tasks.

Important: Datasets

All datasets used in this assignment are available through **NLTK** and **scikit-learn**. No external files need to be downloaded manually. Use the following imports to access them:

- `import nltk` for Brown, Gutenberg, and WebText corpora
- `from sklearn.datasets import fetch_20newsgroups` for the 20 Newsgroups dataset

Make sure to run `nltk.download()` for all required corpora at the top of your notebook.

Part 1: Advanced Smoothing — Good-Turing and Kneser-Ney (25%)

In Assignment 1, you implemented Laplace (add-one) smoothing for bigram language models trained on the Brown Corpus (news + editorial). In this part, you will implement two more sophisticated smoothing techniques **Good-Turing discounting** and **Kneser-Ney smoothing** and rigorously compare them against each other and against your Laplace baseline. These methods address the well-known shortcomings of Laplace smoothing by more intelligently redistributing probability mass.

Helpful Resources

- Jurafsky & Martin, Chapter 3 (N-gram Language Models): https://web.stanford.edu/~jurafsky/slp3/3.pdf
- Good-Turing Smoothing explained: https://en.wikipedia.org/wiki/Good%E2%80%93Turing_frequency_estimation
- Chen & Goodman (1999), "An Empirical Study of Smoothing Techniques for Language Modeling": https://dash.harvard.edu/bitstream/handle/1/25104739/tr-10-98.pdf

Dataset

Training Corpus: Brown Corpus (NLTK) categories `"learned"` and `"government"`. These are formal/academic categories, deliberately different from Assignment 1.

Test Corpus 1 (In-Domain): Brown Corpus (NLTK) — category `"reviews"`. This is formal English but a different topic, serving as an in-domain test.

Test Corpus 2 (Out-of-Domain): NLTK's `webtext` corpus (`nltk.corpus.webtext`). This contains informal internet text (forum posts, movie scripts), providing a strong domain shift.

Load corpora via: `from nltk.corpus import brown, webtext`

Task: P1T1 — Corpus Preparation and Baseline Bigram Model

Prepare the training and test corpora and establish baseline models for comparison.

Steps:

- 1. Load the Brown Corpus categories `"learned"` and `"government"`. Preprocess: lowercase all tokens, add `<s>` and `</s>` sentence markers.
- 2. Report: total number of sentences, total tokens (including markers), vocabulary size (unique word types).
- 3. Train the following **three baseline bigram models**:
  - (a) Unsmoothed bigram model
  - (b) Laplace (add-one) smoothed bigram model
  - (c) Laplace smoothed unigram model (for backoff reference)
- 4. Compute **perplexity** for each baseline model on both test corpora.
- 5. Generate **5 sentences** from each baseline model and save them.

Deliverables:

- Corpus statistics: sentences, tokens, vocabulary size

- Perplexity table: 3 baseline models × 2 test corpora
- 5 generated sentences per model (15 sentences total)
- Brief discussion (2–3 sentences): How do the perplexity values differ between the in-domain (`reviews`) and out-of-domain (`webtext`) test sets? Why?

Task: P1T2 — Good-Turing Smoothing for Bigrams

Implement **Good-Turing discounting** for a bigram language model.

Steps:

- 1. Compute the **frequency of frequencies (N_c)**: for each count c, how many bigrams occurred exactly c times in the training corpus.
- 2. Apply Good-Turing discounting to compute **adjusted counts c*** using the formula:

$c^* = (c + 1) \times N_{(c+1)} / N_c$

- 3. Handle the case where $N_{(c+1)} = 0$ by implementing **Simple Good-Turing (SGT)**: fit a log-linear regression to the (log c, log N_c) data points and use the smoothed N_c values from the regression line when the empirical N_c is unreliable or missing.
- 4. Compute the **total probability mass assigned to unseen bigrams** ($P_o = N_1 / N$, where N is the total number of bigram tokens).
- 5. Implement **sentence generation** and **perplexity computation** using your Good-Turing model.

Deliverables:

- A table showing N_c values for c = 0, 1, 2, 3, 4, 5, 10, 20, 50 and their corresponding adjusted counts c*
- Total probability mass reassigned to unseen bigrams (P_o)
- Compare P_o from Good-Turing vs. Laplace smoothing. Which assigns more mass to unseen events? Explain mathematically why.
- Perplexity values on both test corpora
- 10 generated sentences (saved as good_turing_output.txt)
- Discussion (3–5 sentences): What are the limitations of basic Good-Turing? When does Simple Good-Turing (log-linear regression) break down?

Task: P1T3 — Kneser-Ney Smoothing for Bigrams

Implement **Interpolated Kneser-Ney smoothing** for a bigram language model.

Steps:

- 1. Compute the **discount parameter d** using the formula:

$d = N_1 / (N_1 + 2 \times N_2)$

where N_1 = number of bigrams occurring exactly once, N_2 = number of bigrams occurring exactly twice.

- 2. Implement the **continuation probability P_continuation(w)**: the number of unique bigram types that word w completes (i.e., $|\{w' : C(w', w) > 0\}|$), normalized by the total number of bigram types.
- 3. Implement the full **Interpolated Kneser-Ney formula**:

$P_{KN}(w_i \mid w_{i-1}) = \max(C(w_{i-1}, w_i) - d, 0) / C(w_{i-1}) + \lambda(w_{i-1}) \times P_{continuation}(w_i)$

where $\lambda(w_{i-1}) = (d / C(w_{i-1})) \times |\{w : C(w_{i-1}, w) > 0\}|$

- 4. **Probability verification:** For **5 different conditioning contexts** w_{i-1}, verify that $\sum_w P_{KN}(w \mid w_{i-1})$ sums to 1 (or within floating-point tolerance). Print each sum.
- 5. Implement sentence generation and perplexity computation using your Kneser-Ney model.

Deliverables:

- Computed discount parameter d (with N_1 and N_2 values)
- Show $P_{continuation}$ values for **15 sample words**: include 5 common words, 5 medium-frequency words, and 5 rare words. Explain intuitively why some words have higher continuation probability.
- Probability verification: print the sums for 5 contexts and confirm they equal 1
- For **8 example bigrams** (mix of seen and unseen), show a side-by-side comparison: raw count, Laplace probability, Good-Turing probability, Kneser-Ney probability
- Perplexity values on both test corpora
- 10 generated sentences (saved as kneser_ney_output.txt)

Task: P1T4 — Comprehensive Smoothing Comparison

Perform a rigorous comparison of all smoothing methods.

Steps:

- 1. Compile a perplexity comparison table with all 5 models (unsmoothed bigram, Laplace unigram, Laplace bigram, Good-Turing bigram, Kneser-Ney bigram) on both test corpora.
- 2. **Rank the models** by perplexity for each test corpus. Is the ranking the same for both?
- 3. Compute the **percentage of unseen bigrams** (bigrams in the test corpus that never appeared in training) for each test corpus. How does this correlate with perplexity differences?
- 4. Compare the **generated sentences** qualitatively. Select the 2 most fluent and 2 least fluent sentences across all models. Explain what makes them good or bad.

Deliverables:

- Perplexity comparison table (5 models × 2 test corpora) with model rankings
- Unseen bigram percentages for each test corpus
- Qualitative sentence analysis (2 best, 2 worst, with explanations)
- Discussion (5–8 sentences): Which smoothing method achieves the lowest perplexity and why? How does domain mismatch affect the relative performance of these methods? Why does Kneser-Ney typically outperform both Laplace and Good-Turing?

Part 2: OOV Handling and Naive Bayes Text Classification (25%)

This part explores two fundamental challenges in NLP: handling **out-of-vocabulary (OOV) words** in language models and building a **Naive Bayes classifier** from scratch for **multi-class** text classification. You will implement OOV strategies, analyze their impact on perplexity, and then build and evaluate a probabilistic classifier on a real-world dataset.

Helpful Resources

- Jurafsky & Martin, Chapter 3 (N-gram Language Models OOV handling): https://web.stanford.edu/~jurafsky/slp3/3.pdf
- Jurafsky & Martin, Chapter 4 (Naive Bayes and Sentiment Classification): https://web.stanford.edu/~jurafsky/slp3/4.pdf
- scikit-learn 20 Newsgroups: https://scikit-learn.org/stable/datasets/real_world.html#newsgroups-dataset

Datasets

For OOV tasks: Use the same training corpus from Part 1 (Brown Corpus: `"learned"` + `"government"`). Test on the **Gutenberg corpus** (`nltk.corpus.gutenberg`) use the texts from `"austen-emma.txt"` and `"shakespeare-hamlet.txt"` combined.

For Naive Bayes: Use **scikit-learn's 20 Newsgroups dataset** with the following **4 categories**:

- `"sci.med"`, `"sci.space"`, `"rec.sport.baseball"`, `"talk.politics.guns"`

Load via: `from sklearn.datasets import fetch_20newsgroups`

This is a **4-class classification problem**, more challenging than binary classification. The categories are chosen to have distinct but sometimes overlapping vocabulary.

Task: P2T1 — OOV Analysis and <UNK> Strategies

Analyze the impact of out-of-vocabulary words on language model performance and implement strategies to handle them.

Steps:

- 1. Using the training corpus from Part 1, define a **closed vocabulary** using two different frequency thresholds:
  - (a) Vocabulary V_1: all words appearing **≥ 2 times** in training
  - (b) Vocabulary V_2: all words appearing **≥ 5 times** in training
- 2. For each vocabulary, replace all out-of-vocabulary words in **both training and test data** with `<UNK>`. Retrain your Laplace-smoothed bigram model on the modified training data.
- 3. Compute the **OOV rate** on the Gutenberg test corpus for each vocabulary threshold.
- 4. Compute **perplexity** on the test corpus for each vocabulary threshold. Also compute perplexity **without <UNK> replacement** (open vocabulary) for comparison.
- 5. Identify the **top-30 most frequent OOV words** from the test corpus under each threshold. Categorize them: are they archaic words (Shakespeare), domain-specific terms, proper nouns, or common words missing due to threshold?

Deliverables:

- Training vocabulary size for V_1 and V_2
- OOV rate on test corpus for each threshold
- Perplexity comparison table: open vocabulary vs. V_1 vs. V_2 (3 rows × 1 test corpus)
- Top-30 OOV words for each threshold, categorized by type
- Discussion (5–8 sentences): What is the trade-off between vocabulary size and OOV rate? Why does <UNK> replacement sometimes lower perplexity — is this a genuine improvement or an artifact? How does the mix of Austen and Shakespeare text affect your OOV analysis?

Task: P2T2 — Multinomial Naive Bayes Classifier (From Scratch)

Implement a **Multinomial Naive Bayes** text classifier from scratch. **Do not use sklearn's MultinomialNB, BernoulliNB, or any built-in classifier.** You must implement the algorithm yourself. You may use numpy, collections.Counter, and math for computation.

Steps:

- 1. Load the **20 Newsgroups dataset** (4 categories listed above). Use `fetch_20newsgroups(subset="train", categories=[...])` and `fetch_20newsgroups(subset="test", categories=[...])` — the dataset provides a standard train/test split.
- 2. Preprocess: lowercase, remove punctuation, remove stopwords, tokenize. Remove any email headers, footers, and quoted text from the posts.
- 3. Implement the Multinomial Naive Bayes classifier:
    - (a) Compute **prior probabilities**: P(c) = count(docs in class c) / total_docs
    - (b) Compute **class-conditional likelihoods**: P(w | c) with **Laplace (add-one) smoothing**: P(w | c) = (count(w in class c) + 1) / (total_words_in_c + |V|)
    - (c) For prediction, compute: log P(c | d) = log P(c) + Σ log P(w | c) for each class, predict the class with highest score
- 4. Report **accuracy, precision, recall, and F1-score** both **per-class** and **macro-averaged**.
- 5. Compute and display the **confusion matrix**.
- 6. For each class, identify the **top-15 most informative features**: words with the highest P(w | c) / P(w | not c) ratio.

Deliverables:

- Per-class and macro-averaged metrics: accuracy, precision, recall, F1
- Confusion matrix (4×4)
- Top-15 most informative features per class (4 classes × 15 words)
- Show **4 misclassified documents** (one per class if possible) print a portion of the text and the predicted vs. true label. Analyze why the classifier failed.
- Discussion (3–5 sentences): Which class is easiest to classify and which is hardest? Why? How does class overlap in vocabulary affect performance?

Task: P2T3 — Ablation Study and Baseline Comparisons

Systematically evaluate your Naive Bayes classifier through ablation studies and baseline comparisons.

Steps:

- 1. Implement two baselines:
  - (a) **Majority-class baseline**: always predict the most frequent class
  - (b) **Random-weighted baseline**: predict classes with probability proportional to their training set frequency
- 2. Compare all three models (Naive Bayes, majority-class, random-weighted) using accuracy and macro-F1.
- 3. **Learning curve**: train your Naive Bayes on 10%, 20%, 40%, 60%, 80%, and 100% of the training data. Plot accuracy and macro-F1 vs. training set size.
- 4. **Smoothing ablation**: vary the Laplace smoothing parameter $\alpha \in \{0.01, 0.1, 0.5, 1.0, 2.0, 5.0\}$ and report accuracy for each. Plot α vs. accuracy.
- 5. **Feature selection**: try removing the top-50 most common words (beyond stopwords) from the vocabulary. Does this improve or hurt performance?

Deliverables:

- Comparison table: Naive Bayes vs. baselines (accuracy and macro-F1)
- Learning curve plot (accuracy and macro-F1 vs. training size)
- Smoothing parameter plot (α vs. accuracy) what is the optimal α?
- Feature selection results: accuracy before and after removing top-50 common words
- Discussion (5–8 sentences): At what training size does Naive Bayes plateau? What does the optimal α tell you about the dataset? Why might removing very common words help or hurt?

Deliverables

1. 1 Notebook

- A single Jupyter notebook (.ipynb) containing code implementations for all parts, organized sequentially by the specified TASK IDs.
- All results must be visible and reproducible, with every cell already executed.
- The notebook must not require re-running to verify outputs.
- All datasets must be loaded programmatically (via NLTK or sklearn) no external files.

2. 1 Detailed Report

- A well-structured written report covering each task separately.
- Include screenshots of results and outputs for every task.
- All discussion questions must be answered thoroughly with evidence from your results.
- Ensure proper academic formatting, clear headings, and consistent labeling.

3. Exported Files

- All required exported artifacts: generated sentence files (good_turing_output.txt, kneser_ney_output.txt), plots (learning curves, parameter plots), and any other output files referenced in the notebook and report.

Screenshots in the report should correspond to key results and tables shown in the notebook.

Important Notes

- Incomplete execution, missing TASK IDs, or improperly formatted reports may result in loss of marks.
- Ensure consistency between notebook outputs, screenshots, and exported files.
- **"From scratch" implementation:** Do NOT use sklearn.naive_bayes.MultinomialNB or BernoulliNB for the Naive Bayes classifier. You must implement the algorithm yourself. You may use: numpy, math, collections.Counter, sklearn.metrics for evaluation metrics.
- All NLTK corpora used must be downloaded at the top of your notebook to ensure reproducibility.